

Advanced Tools for Complex Network Analysis

CNET 5052, Spring 2026

Prof. Brennan Klein and Dr. Milo Trujillo

Assignment 2, due by Friday, February 20, 2026 (a.o.e.)

Please submit a .pdf of your answers via Canvas, including a url to your .ipynb notebook in your fork of our class repository. Please name your Canvas submission as Lastname.pdf, (e.g. Klein.pdf). If you need help with LaTeX or GitHub, please visit us during office hours or post questions to the Slack channel.

AI Tags. Each question below has a tag indicating what kind of AI usage is appropriate, if at all:

- ✧ AI-FORBIDDEN ✧ : Do not use generative AI for this. We want your own thinking and voice.
- ✧ AI-LIMITED ✧ : You may use AI for debugging help, documentation help, or short clarifications. Do not paste in complete solutions. You must still understand and be able to explain your work.
- ✧ AI-PERMITTED ✧ : You may use AI as a helper (including code suggestions), but you must cite it, verify by running and testing, and write your own explanations.
- ✧ AI-ENCOURAGED ✧ : You are encouraged to use AI as a learning partner, if helpful. These parts are meant to build programming confidence and grasp.

In this course, the use of AI tools is experimental and intended to enhance your work and learning. Please use this privilege wisely, as we reserve the right to revoke this policy.

0. Question 0 (5 points).

✧ AI-ENCOURAGED ✧

Install the `graph-tool` Python package, using the installation instructions/tips here: <https://graph-tool.skewed.de/>. We recommend trying to install using Conda, via `conda-forge`. The packages are binary, so no compilation is necessary. You may also find it helpful to explore the tutorials and Quick Start pages to get a feel for the syntax and common usage of this extensive package.

1. Bayesian Statistics and Networks (25 points).

✧ AI-LIMITED ✧

Two adjacency lists are provided with this assignment: `mystery_edges1.txt` and `mystery_edges2.txt`. Each line of the file represents the edges from a vertex (i.e. the second row represents all the vertices that node 1 has edges to). These networks *may* have come from the Barabási-Albert model, where each node adds m edges to the graph, and has the following probability of connecting to each neighbor:

$$p_i = \frac{k_i}{\sum_j k_j}$$

Your goal is to determine whether the provided graphs are likely to have been generated by the above model. For simplicity, you may assume that the networks come from *some* growth model, and that nodes were added in order (i.e. node 10 was added to the network after node 9). This will allow you to reproduce the degree sequence that existing at the time that any node was added if needed.

- (a) Fit the growth model to each provided network to determine the most likely m value.
- (b) Generate an ensemble of one hundred networks with the fit model. These ensembles should have approximately the same number of nodes and edges as the provided network.
- (c) Choose three graph-level attributes. Plot a distribution (kernel-density estimates and cumulative distribution functions are good choices here) of each graph-level attribute for each graph ensemble. Add a reference line for the mystery network so we can compare its attributes to the ensemble.
- (d) For each mystery network, can we reject the null-hypothesis that the network was created using the above network growth model? Why or why not?

Note: This closely mirrors an in-class exercise where we fit ER models to provided data. If you are feeling lost on steps or what we are looking for in terms of visualization, check the lecture notebook.

2. Replication: Modularity in random graphs (15 points).

✧ AI-PERMITTED ✧

Recall in `class_04`, we looked at modularity in random (i.e., $G(n, m)$) graphs. Expand this result by replicating Figure 2 from Guimera, R., Sales-Pardo, M., & Amaral, L.A.N. (2004). Modularity from fluctuations in random graphs and complex networks. *Physical Review E*, 70(2), 025101. Comment on this result, especially as it relates to modularity as a measure of partition quality.

3. Exploring community detection (30 points).

✧ AI-LIMITED ✧

This question is designed to build more intuition around what exactly is happening in community detection. What does it mean to detect communities? What does the partition signify? What biases might we find?

- (a) Download a network dataset from <https://networkrepository.com/> with between $1,000 < N < 5,000$ nodes. Select a modularity maximization algorithm to detect communities in this graph.

- i. Visualize your network, coloring the nodes by community membership.
 - ii. Report the modularity of the partition your algorithm found.
 - iii. Report the number of communities detected.
 - iv. Store your partition as a dictionary, in the form of `{node_id: community_id}`.
- (b) Using the same network as in (a), use `graph-tool` to create a partition of your network (i.e., run community detection). Visualize your network again, coloring the nodes by community membership; use the same network visualization layout as the in (a). Store your partition as a dictionary
- (c) Create a degree-preserving randomization of your network, and run the two community detection algorithms from above on your randomized graph. Store each partition as a dictionary.
- (d) You should now have four different dictionaries, corresponding to the partitions created in (a-c). Create a figure with four subplots: In each, plot the *community size distribution* of each partition. Your figure should be publication-ready quality, with well-labeled, consistent axes, log-scaled where relevant, etc.
- (e) For the different partitions computed above, discuss how you would approach comparing these partitions quantitatively. For example, imagine a function $f(b_1, b_2)$ that takes two partitions (b_1 and b_2) as input and returns a number corresponding to how similar/dissimilar the partitions are. What properties should this function f have? Base your answer on a technique used in the literature, after a brief search about methods for quantifying differences between graph partitions.

4. Community detection and the consensus partition (25 points).

✧ AI-PERMITTED ✧

Many community detection algorithms are stochastic: their output can change across runs due to random seeds, initialization, or tie-breaking. In this problem, you will implement a basic *consensus community detection* pipeline and compare it to both a single stochastic run and a deterministic baseline.

- (a) Do a brief literature search for *consensus clustering* in networks (or *consensus community detection*) in networks. Define the idea in your own words, define what a *consensus matrix* is, cite at least one peer-reviewed source.
- (b) Generate a network from an undirected microcanonical stochastic block model (a “planted-partition” model) graph with four assortative communities of size 100, 200, 300, 400 ($N = 1,000$). Report the within-block and between-block connection probabilities that you used, and store the planted community labels as a dictionary `{node_id: planted_block}`.
- (c) Choose a *stochastic* algorithm of your choice (e.g., Louvain/Leiden, label propagation, etc.; there are many). Run this algorithm $R = 100$ times on your graph, using different random seeds. For each run r , store the partition as a dictionary `{node_id: community_id}` and record the modularity and the number of detected communities. Create a figure with two subplots that visualize the 1) distribution

of modularity values of your partitions, and 2) the distribution of number of communities across runs.

- (d) Let $b^{(r)}(i)$ be the community label of node i in run r . Implement the consensus matrix C defined by

$$C_{ij} = \frac{1}{R} \sum_{r=1}^R \mathbb{1}\{b^{(r)}(i) = b^{(r)}(j)\}. \quad (1)$$

Create a visualization of C (using `plt.imshow()` or `plt.matshow()`).

- (e) Convert C into a *consensus graph* (weighted), with the nodes of C being the same nodes as your original graph, and give edge (i, j) weight C_{ij} . Because C will likely be typically dense, apply a filtering rule: choose a threshold τ and discard edges with $C_{ij} < \tau$. State and justify your choice of τ . Finally (!), run a community detection algorithm on the *thresholded* consensus graph to get a final partition $b^{(\text{cons})}$. (You may use the same stochastic algorithm with a fixed seed, or a deterministic weighted-community method.)
- (f) Repeat steps (b)-(e), *with less strong-community structure*. This means the within-block probabilities (i.e., your p_{aa} , p_{bb} , p_{cc} , etc.) should be smaller this time, and/or your between-block probabilities should be larger.
- (g) Across your experiments, does the consensus partition help improve the community detection process? Under what conditions did it help, if any? Discuss.